

Dossier de remise de l'app

BookCollection

Rappel du sujet

L'objectif de ce projet était de concevoir un outil de gestion de bibliothèque personnelle. Aujourd'hui, un lecteur possède souvent une collection mixte, composée à la fois de livres physiques (papier) et de fichiers numériques (PDF, EPUB). Le problème principal est qu'il est difficile de centraliser tous ces formats au même endroit et de suivre précisément sa progression de lecture. De plus, faire l'inventaire de sa bibliothèque en tapant tout manuellement est une tâche longue et décourageante. J'ai donc voulu créer une solution qui simplifie tout ce processus.

Pour répondre à ce besoin, j'ai développé une application mobile articulée autour de fonctionnalités pratiques :

- **L'ajout par scan de code-barres** : Pour éviter la saisie manuelle, l'utilisateur peut utiliser la caméra de son smartphone pour scanner l'ISBN d'un livre physique. L'application fait appel à l'API publique de Google Books pour récupérer automatiquement toutes les informations (titre, auteur, couverture, résumé, nombre de pages). Si le livre n'est pas reconnu, un formulaire permet l'ajout ou la recherche manuelle.
- **Le suivi de lecture et les statistiques** : Pour chaque livre, l'utilisateur peut indiquer la page à laquelle il s'est arrêté. L'application met à jour le statut (*À lire*, *En cours*, *Terminé*) et alimente un écran de statistiques globales affichant le nombre total de pages lues, de livres possédés et la répartition des formats.
- **Un lecteur de fichiers intégré** : L'utilisateur peut associer un fichier (PDF ou EPUB) stocké sur son téléphone à un livre numérique de l'application. J'ai implémenté une double approche : les PDF s'ouvrent directement dans l'application via une vue web intégrée, tandis que les EPUBs sont délégués au système natif du téléphone pour garantir le meilleur affichage possible.

Spécification technique

Module 1 : Gestion de la Bibliothèque (CRUD local)

- **Affichage de la collection** : Liste de tous les livres enregistrés avec un aperçu rapide (titre, auteur, format).

- **Fiche détaillée** : Consultation des informations complètes d'une œuvre (couverture, résumé complet, statut).
- **Modification (Update)** : Possibilité d'éditer les champs d'un livre existant via un formulaire pré-rempli.
- **Suppression (Delete)** : Retrait d'un livre de la base de données, sécurisé par une fenêtre d'alerte native de confirmation pour éviter les erreurs de manipulation.
- **Sécurité anti-doublon** : Vérification automatique lors de l'ajout pour interdire l'enregistrement d'un même livre dans un format identique (ex: impossible d'ajouter deux fois *L'Art de la Guerre* en format "Physique", mais possible d'avoir une version "Physique" et une version "Numérique").

Module 2 : Acquisition des données (3 méthodes d'ajout)

- **Smart Scan (Code-barres)** : Utilisation de l'appareil photo pour scanner un ISBN. L'application interroge l'API Google Books et pré-remplit instantanément toutes les métadonnées de l'œuvre.
- **Recherche en ligne** : Un moteur de recherche textuel connecté à Google Books permettant de trouver un livre par son titre ou son auteur (avec gestion des alertes si la recherche est trop courte ou ne donne aucun résultat).
- **Saisie manuelle** : Un formulaire de création complet permettant d'ajouter une œuvre indépendante, non répertoriée sur internet.

Module 3 : Suivi de Lecture (Tracking & Statistiques)

- **Mise à jour de la progression** : Un champ permet à l'utilisateur de renseigner sa page actuelle.
- **Calcul intelligent du statut** : L'application met à jour le statut du livre automatiquement ("À lire" si page 0, "En cours" au-delà, et "Terminé" si la page actuelle atteint ou dépasse le nombre de pages total).
- **Tableau de bord (Dashboard)** : Un écran de statistiques globales généré dynamiquement qui agrège les données de la base : nombre total de pages lues, livres terminés vs en cours, et répartition des formats (Physique vs Numérique).

Module 4 : Gestion des Ebooks (Lecture Numérique)

- **Liaison de fichiers** : L'utilisateur peut parcourir la mémoire de son téléphone pour associer un fichier (PDF ou EPUB) à une fiche de livre "Numérique".
- **Lecteur PDF intégré** : Ouverture des fichiers PDF directement au sein de l'application via une WebView.
- **Délégation d'ouverture (EPUB)** : Pour les formats complexes comme l'EPUB, l'application utilise le système de partage natif (Share API) de l'OS (iOS ou Android) pour proposer à l'utilisateur de l'ouvrir dans son application de lecture dédiée.

Justification technique

1. React Native, Expo et TypeScript

- **React Native & Expo** : Ce choix m'a permis de développer une application cross-platform (iOS et Android) avec une seule base de code. Expo a grandement facilité

l'accès aux API natives de l'appareil (Caméra, Système de fichiers, Partage) sans avoir à configurer manuellement les fichiers natifs complexes (Podfile, Gradle). J'ai également utilisé **Expo Router** pour bénéficier d'une navigation basée sur les fichiers (à la manière de Next.js), ce qui rend la structure de l'application très lisible.

- **TypeScript** : Bien que le JavaScript classique soit plus permissif, j'ai imposé l'utilisation de TypeScript. Le typage fort (interfaces pour les Livres, pour les retours de l'API Google, etc.) m'a permis d'éviter de nombreux bugs à l'exécution (runtime errors)

2. Base de données : SQLite plutôt que AsyncStorage ou Firebase

- Mon objectif était de créer une application "Offline-First" (fonctionnant sans internet) et respectueuse de la vie privée. Firebase (Cloud) n'était donc pas adapté.
- Pour le stockage local, j'aurais pu utiliser *AsyncStorage*. Cependant, ce dernier n'est qu'un magasin "clé-valeur", inadapté pour faire des recherches complexes, des filtres ou des statistiques. Le choix d'**expo-sqlite** s'est imposé : c'est une vraie base de données relationnelle. Cela m'a permis d'utiliser des requêtes SQL natives performantes (SELECT, UPDATE, DELETE) et de garantir l'intégrité de mes données.

3. Appels réseau : L'API Fetch (Natif) vs Axios

- Pour communiquer avec l'API Google Books, j'ai fait le choix d'utiliser l'API native **fetch** de JavaScript, et non la bibliothèque très populaire *Axios*.
- **Pourquoi ?** L'application ne réalise que de simples requêtes GET publiques (sans gestion complexe de tokens d'authentification dynamiques ou d'intercepteurs). Installer *Axios* aurait alourdi inutilement le poids de l'application (*bundle size*). L'API fetch standard, couplée à des blocs try/catch pour la gestion des erreurs HTTP (ex: erreur 400), s'est révélée amplement suffisante et plus légère.

Architecture

1. Structure des dossiers (Le principe de séparation)

L'arborescence du projet est divisée en plusieurs couches distinctes :

- **app/ (Les Vues / Le Routeur)** : C'est le point d'entrée visuel de l'application, géré par *Expo Router*. Ces fichiers (`_layout.tsx`, `index.tsx`, `stats.tsx`, `book/[id].tsx`) ne contiennent que du code d'affichage (JSX) et du design (CSS/StyleSheet). Les Vues sont "bêtes" : elles ne font aucun calcul complexe, elles se contentent d'appeler les contrôleurs et d'afficher les données qu'on leur donne.
- **src/components/ (Les Composants réutilisables)** : Ce dossier contient les éléments d'interface isolés (comme `book-form.tsx`, `themed-text.tsx`). Cela permet de respecter le principe DRY (*Don't Repeat Yourself*) en réutilisant les mêmes briques graphiques à travers toute l'application.

- **src/hooks/ (Les Contrôleurs)** : C'est le "cerveau" de l'application. Ces fichiers (ex: useScannerController.ts, useStatsController.ts) contiennent les Hooks personnalisés. Ils font le pont entre la Vue et le Modèle. Ils gèrent l'état local (state), exécutent la logique métier (vérification des doublons, formatage des données) et orchestrent les actions de l'utilisateur.
- **src/services/ (Les Modèles)** : C'est la couche d'accès aux données.
 - bookService.ts s'occupe exclusivement d'écrire et de lire des requêtes SQL dans la base SQLite locale.
 - googleBookService.ts s'occupe exclusivement de faire les requêtes HTTP (Fetch) vers l'API de Google et de formater le JSON reçu.

Directory structure:

```

└─ BookCollection/
   ├── README.md
   ├── app.json
   ├── eslint.config.js
   ├── package.json
   ├── tsconfig.json
   └── app/
       ├── _layout.tsx
       ├── modal.tsx
       ├── (tabs)/
       │   ├── _layout.tsx
       │   ├── index.tsx
       │   ├── scanner.tsx
       │   └── stats.tsx
       └── book/
           └── [id].tsx
   ├── constants/
   │   └── theme.ts
   ├── hooks/
   │   ├── use-color-scheme.ts
   │   ├── use-color-scheme.web.ts
   │   └── use-theme-color.ts
   ├── scripts/
   │   └── reset-project.js
   └── src/
       ├── components/
       │   ├── book-card.tsx
       │   ├── book-form.tsx
       │   ├── external-link.tsx
       │   ├── haptic-tab.tsx
       │   ├── hello-wave.tsx
       │   ├── parallax-scroll-view.tsx
       │   ├── themed-text.tsx
       │   ├── themed-view.tsx
       │   └── ui/
       │       ├── collapsible.tsx
       │       ├── icon-symbol.ios.tsx
       │       └── icon-symbol.tsx
       ├── database/
       │   └── config.ts
       ├── hooks/
       │   ├── useManualSearch.ts
       │   ├── useScannerController.ts
       │   └── useStatsController.ts
       └── services/
           ├── bookService.ts
           └── googleBookService.ts

```

2. Flux de données

- **Affichage de la bibliothèque (Écran principal)**
- **Chemin:** app/(tabs)/index.tsx (Vue)
 - src/services/bookService.ts (Service/Modèle) → Base de données SQLite.
 - Le fichier d'interface demande directement au service de lire la base de données et affiche la liste.*
- **Scan d'un code-barres et enregistrement**
- **Chemin:** app/(tabs)/scanner.tsx (Vue)
 - src/hooks/useScannerController.ts (Contrôleur)
 - src/services/googleBookService.ts (Service externe)
 - src/services/bookService.ts (Service local) → SQLite.
 - La vue capte le code, le contrôleur l'envoie au service Google pour récupérer les infos, puis demande au service local de vérifier les doublons et de sauvegarder.*
- **Recherche manuelle en ligne**
- **Chemin :** app/modal.tsx (Vue) → src/hooks/useManualSearch.ts (Contrôleur)
 - src/services/googleBookService.ts (Service externe).
 - L'utilisateur tape son texte dans la vue, le contrôleur valide la requête (min. 3 caractères) et interroge l'API Google, puis renvoie la liste à la vue.*
- **Sauvegarde d'un livre recherché manuellement (ou créé de zéro)**
- **Chemin :** app/modal.tsx (Vue) → src/components/book-form.tsx (Composant UI)
 - src/services/bookService.ts (Service local) → SQLite.
 - Le formulaire valide les champs saisis et envoie directement l'ordre d'écriture au service de base de données.*
- **Affichage des détails d'un livre**
- **Chemin :** app/book/[id].tsx (Vue) → src/services/bookService.ts (Service local) → SQLite.
 - Au clic sur un livre, la page récupère l'ID dans l'URL et demande au service local d'aller chercher la ligne correspondante en base.*
- **Mise à jour ou Suppression d'un livre**
- **Chemin:** app/book/[id].tsx (Vue) → src/components/book-form.tsx (Si modification) → src/services/bookService.ts (Service local) → SQLite.
 - Les actions de l'utilisateur (bouton "Supprimer" ou validation du formulaire de modification) déclenchent les requêtes SQL UPDATE ou DELETE dans le service.*
- **Suivi de progression (Mise à jour des pages lues)**
- **Chemin :** app/book/[id].tsx (Vue) → src/services/bookService.ts (Service local) → SQLite.
 - La vue calcule le nouveau statut (En cours / Terminé) selon le nombre de pages entré et envoie l'instruction de mise à jour au service.*

- **Affichage du Tableau de bord (Statistiques)**
- **Chemin:** app/(tabs)/stats.tsx (Vue) → src/hooks/useStatsController.ts (Contrôleur) → src/services/bookService.ts (Service local) → SQLite.
La vue demande les stats. Le contrôleur récupère tous les livres via le service, fait les calculs mathématiques (pages totales, répartition), et renvoie le résultat formaté à la vue.

Auto-évaluation

Fonctionnalité annoncée	État de réalisation	Remarques et détails techniques
Base de données locale	✓ Réalisé	Implémentation réussie avec expo-sqlite. Opérations CRUD complètes et 100 % hors-ligne.
Ajout par scan de code-barres	✓ Réalisé	Intégration de la caméra (expo-camera) et liaison avec l'API Google Books.
Ajout manuel / Recherche	✓ Réalisé	Création d'un formulaire manuel (react-hook-form) et d'un moteur de recherche par titre/auteur.
Tableau de bord (Statistiques)	✓ Réalisé	Création d'un écran de statistiques globales dynamiques optimisé avec useMemo.
Gestion des erreurs (UX)	✓ Réalisé	Prévention des doublons en base de données et implémentation d'alertes natives.
Comptage dynamique des pages	⚠ Partiel	Remplacé par une saisie manuelle assistée (le statut se met à jour automatiquement selon la page entrée).
Lecteur EPUB interne	⚠ Partiel	Remplacé par une délégation au système de l'appareil via la Share API.

Configuration particulières

Pour utiliser l'API de google book, j'ai récupéré une clé API que j'ai mis dans un .env qui est à la racine du projet.

Le .env est composé de :

```
EXPO_PUBLIC_GOOGLE_BOOK_API_KEY=MaCleSecrete
```

Les différentes étapes pour récupérer la clé :

Étape 1 : Accès à la Google Cloud Console

1. Ouvre un navigateur web et se rendre sur [Google Cloud Console](#).
2. Se connecter avec un compte Google.
3. Accepte les conditions d'utilisation si c'est première visite

Étape 2 : Création du projet

1. Dans la barre de navigation en haut, cliquer sur le menu déroulant des projets.
2. Dans la fenêtre qui s'ouvre, cliquer sur le bouton **Nouveau projet** en haut à droite.
3. Donner un nom explicite au projet (par exemple : BookCollection-App) et clique sur **Créer**.
4. Attendre quelques secondes, puis sélectionner ce nouveau projet via l'icône de notification (en forme de cloche) ou le menu déroulant en haut.

Étape 3 : Activation de l'API Google Books

1. Ouvrir le menu de navigation principal (les trois lignes horizontales en haut à gauche).
2. Aller dans **API et services**, puis cliquer sur **Bibliothèque**.
3. Dans la barre de recherche, taper Books API et valider.
4. Cliquer sur le premier résultat ("Books API").
5. Cliquer sur le bouton bleu **Activer**.

Étape 4 : Génération de la clé API

1. Une fois l'API activée, retourner dans le menu de gauche et sélectionne **API et services > Identifiants**.
2. En haut de l'écran, cliquer sur le bouton **+ CRÉER DES IDENTIFIANTS**.
3. Dans le menu déroulant, sélectionner **Clé API**.

Étape 5 : Sécurisation de la clé (Configuration du menu de restrictions)

1. **Nom de la clé (en haut) :** Il est recommandé de renommer la clé (par exemple en "Clé App BookCollection") pour la distinguer facilement sur le tableau de bord Google Cloud.
2. **Restrictions d'API (au milieu) :** Dans le menu déroulant "Sélectionner des restrictions d'API", il faut chercher et cocher **Books API**.

3. **Restrictions relatives aux applications (en bas) :** Laisser sur aucun
4. Cliquer sur le bouton **Enregistrer** tout en bas de la page pour appliquer cette politique de sécurité.

Étape 6 : Intégration dans le projet React Native (Expo)

1. Ouvrir ton projet dans ton éditeur de code (ex: VS Code).
2. À la racine absolue du projet (au même niveau que le fichier package.json), créer un fichier nommé exactement .env.
3. Ouvrir ce fichier et déclarer la variable d'environnement en utilisant obligatoirement le préfixe d'Expo :
`EXPO_PUBLIC_GOOGLE_BOOK_API_KEY=clésecrète`